

ArcVault Triage Pipeline: Prompt Documentation

Overview

The pipeline uses a single LLM call per message. GPT-4o receives a system prompt that defines the classification task and output schema, and a user prompt that injects the raw message. Everything comes back in one JSON object: the category, priority, confidence score, extracted entities, a one-sentence core issue, and a human-readable summary.

System Prompt

You are an intake triage engine for ArcVault, a B2B software company. You read one raw customer message and return a single structured JSON object.

Be decisive but calibrate your confidence honestly: lower it when the message is ambiguous, mixes multiple intents, or lacks detail.

Classify into exactly one category:

- Bug Report: something is broken for this user (errors, failures, unexpected behavior).
- Incident/Outage: a service is down or degraded, especially affecting multiple users.
- Feature Request: asking for new or expanded functionality.
- Billing Issue: invoices, charges, pricing, refunds.
- Technical Question: how-to, configuration, integration, or evaluation questions.

Priority guidance:

- High: outages, blocked logins, anything affecting multiple users or money.
- Medium: single-user bugs, billing discrepancies, time-sensitive questions.
- Low: feature ideas, general curiosity, no urgency.

Output ONLY a valid JSON object in exactly this shape, no prose, no markdown:

```
{
  "category": "Bug Report | Incident/Outage | Feature Request | Billing
Issue | Technical Question",
  "priority": "Low | Medium | High",
  "confidence": 0.0,
  "core_issue": "one sentence stating the underlying problem",
  "entities": {
    "account_ids": [],
    "invoice_numbers": [],
    "error_codes": [],
    "amounts": [],
    "excess_amount": null,
    "products_or_systems": [],
    "timeframe": ""
  },
}
```

```
    "urgency_signal": "short phrase explaining the urgency level you
assigned",
    "summary": "2-3 sentence human-readable summary for the receiving
team"
}
```

User Prompt

```
Source: {{source}}
Message:
"""
{{raw_message}}
"""
Return the JSON object now.
```

{{source}} and {{raw_message}} are injected at runtime by n8n using the values from the Normalize node.

Design rationale

Why one combined call

Classification, entity extraction, and the summary all happen in a single call rather than separate steps. This keeps latency low, minimises cost, and reduces the number of failure points. The tradeoff is that the model has to satisfy multiple output requirements at once, which can occasionally cause it to trade off quality in one area to satisfy another. A two-call split, structured fields first and summary second, would eliminate that tradeoff at a modest cost increase. That is the first thing I would change in a production version.

Why confidence is framed as a calibration instruction

The prompt tells the model to lower confidence when the message is ambiguous or mixes intents, rather than asking how sure are you. The distinction matters: the second framing invites the model to be generically confident, while the first gives it a concrete reason to be honest. This makes the 0.70 escalation threshold meaningful. If the model returns 0.85 on a clear bug report and 0.55 on an ambiguous message, those scores are actually useful for routing. If it always returns 0.9, the threshold does nothing.

Why the JSON schema is included in the prompt

GPT-4o's JSON mode works most reliably when a sample of the target structure is provided alongside the word JSON. Without the schema example, the model sometimes invents field names or returns a different shape. Including the full skeleton with placeholder values locks the output shape and means the downstream parser never has to guess whether a field exists.

Why the allowed values are spelled out as enums

Listing the exact strings for category and priority prevents the model from returning variations like "bug" instead of "Bug Report", or "critical" instead of "High". Combined with the validation step in the Parse + Validate node, which rejects any value not in the

allowed set and falls back to a safe default, the pipeline is robust to minor model drift between runs.

Why `excess_amount` is a separate entity field

The billing escalation rule checks whether a discrepancy exceeds \$500. If a message contains two dollar figures, the routing node subtracts them. But if a user writes "you charged me \$800 extra", there may be only one extractable amount, so the subtraction approach fails. The `excess_amount` field tells the model to extract the stated overcharge directly when the user phrases it that way, so the routing node has a reliable number to check regardless of how the complaint was expressed.